

Exam 20-12-2024 - Selected Solutions

Problem 1.1: Directed Largest Common Subgraph is NP-Complete

1. Appartenenza a NP: Il certificato (witness) è una tupla $t = (E'_1, E'_2, f)$, dove $E'_1 \subseteq E_1$, $E'_2 \subseteq E_2$ ed $f: V_1 \rightarrow V_2$ è una funzione biiettiva esplicita. Un verificatore può controllare in tempo polinomiale $O(|V_1| + |E_1| + |E_2|)$ che: * $|E'_1| = |E'_2| \geq k$ * f sia una biiezione valida tra V_1 e V_2 * Per ogni arco $(u, v) \in E'_1$, l'arco corrispondente $(f(u), f(v))$ esista in E'_2 .

2. NP-Hardness (Riduzione da Directed Hamiltonian Cycle): * **Costruzione:** Data un'istanza generica $G = (V, E)$ con n nodi. Definiamo $G_1 = G$. Costruiamo G_2 come un singolo ciclo diretto semplice di n vertici. Impostiamo il parametro $k = n$. * **Dimostrazione:** * ($\Rightarrow$) Se G ha un ciclo hamiltoniano, possiamo scegliere quegli n archi come E'_1 , e tutti gli n archi di G_2 come E'_2 . La biiezione f mappa l'ordine del ciclo in G_1 sul ciclo G_2 . * ($\Leftarrow$) Se esiste una soluzione, G'_2 deve usare tutti i suoi n archi (essendo $k = n$), formando un ciclo di n nodi. Poiché $G'_1 \cong G'_2$, anche gli n archi scelti in G_1 formano un ciclo orientato di n nodi, ovvero un ciclo hamiltoniano per G .

Problem 1.2: Dominating Set is NP-Complete

Riduzione da Vertex Cover: * **Costruzione:** Data un'istanza di Vertex Cover (G, k) senza vertici isolati. Costruiamo G' mantenendo tutti i vertici e gli archi di G . Per ogni arco $e = (u, v)$, aggiungiamo un nodo ausiliario w_e e due archi (u, w_e) e (v, w_e) , creando un triangolo. Il parametro rimane $k' = k$. * **Dimostrazione:** * ($\Rightarrow$) Se S è un Vertex Cover di taglia k , per ogni arco $e = (u, v)$ almeno un estremo è in S . Questo estremo è adiacente a w_e , quindi lo domina. Poiché non ci sono nodi isolati, tutti i nodi in V e tutti i w_e sono dominati da S . * ($\Leftarrow$) Se S' è un Dominating Set di taglia k in G' , per dominare w_e deve contenere almeno un nodo tra u, v, w_e . Se contiene w_e , possiamo scambiarlo con u (o v) mantenendo inalterata la dominazione. Ripetendo questo per tutti i nodi ausiliari, otteniamo un Dominating Set composto solo da vertici originali. Essendo ogni w_e dominato, questo insieme include almeno un estremo di ogni arco originale, rivelandosi un Vertex Cover valido di taglia k .

Problem 2: Min-Edge Coloring (2-Approximation)

1. Lower Bound: Sia Δ il grado massimo del grafo. Poiché gli archi incidenti su un singolo nodo devono avere colori tutti distinti, la soluzione ottima richiede almeno Δ colori. Dunque $OPT \geq \Delta$.

2. Algoritmo Greedy: Scorriamo gli archi in un ordine arbitrario. Per ogni arco $e = (u, v)$, gli assegniamo il più piccolo colore non ancora utilizzato da nessun altro arco incidente su u o su v .

3. Analisi del Rapporto d'Approssimazione: Quando coloriamo un arco (u, v) , ci sono al massimo $\deg(u) - 1 \leq \Delta - 1$ altri archi incidenti su u , e al massimo $\deg(v) - 1 \leq \Delta - 1$ altri archi incidenti su v . Il numero totale di colori bloccati (conflitti) per l'arco corrente è al massimo:

$$(\Delta - 1) + (\Delta - 1) = 2\Delta - 2$$

L'algoritmo Greedy userà quindi, nel peggiore dei casi, il colore successivo disponibile:

$$ALG \leq (2\Delta - 2) + 1 = 2\Delta - 1 \leq 2\Delta$$

Confrontando con il lower bound:

$$\frac{ALG}{OPT} \leq \frac{2\Delta}{\Delta} = 2$$

L'algoritmo restituisce una 2-approssimazione.

Problem 3: Max-3-Hypergraph-Cut (Randomized Approximation)

1. Algoritmo Randomizzato: Scorriamo tutti i nodi del grafo $G = (V, E)$ e assegniamo ciascun nodo $v \in V$ alla partizione V_1 o alla partizione V_2 in modo indipendente e uniforme, con probabilità $p = \frac{1}{2}$ per ciascuna scelta [cite: 6].

2. Analisi della singola iper-clausola: Ogni iperarco (hyperedge) $e \in E$ è composto da esattamente 3 vertici, $e = u, v, z$ [cite: 6]. Poiché le assegnazioni dei vertici sono indipendenti, ci sono $2^3 = 8$ configurazioni possibili per i tre nodi. L'iperarco *non* viene tagliato solo in 2 di queste configurazioni: * Tutti i 3 nodi finiscono in V_1 * Tutti i 3 nodi finiscono in V_2 . Nei restanti 6 casi, l'iperarco ha almeno un nodo in V_1 e almeno un nodo in V_2 , risultando tagliato [cite: 6]. La probabilità che un generico iperarco e venga tagliato è dunque:

$$\Pr[X_e = 1] = \frac{6}{8} = \frac{3}{4}$$

3. Rapporto di Approssimazione (Linearità del Valore Atteso): Sia X_e la variabile indicatrice che vale 1 se l'iperarco e è tagliato e 0 altrimenti. Il valore atteso del numero di iperarchi tagliati dall'algoritmo (ALG) si ottiene tramite la linearità del valore atteso calcolando la somma su tutti gli iperarchi:

$$\mathbb{E}[ALG] = \sum_{e \in E} \mathbb{E}[X_e] = \sum_{e \in E} \frac{3}{4} = \frac{3}{4} |E|$$

Poiché la soluzione ottima (OPT) non può tagliare un numero di iperarchi superiore al numero totale di iperarchi presenti nel grafo, abbiamo il limite superiore $OPT \leq |E|$. Sostituendo questa disuguaglianza, otteniamo:

$$\mathbb{E}[ALG] = \frac{3}{4} |E| \geq \frac{3}{4} OPT$$

Questo dimostra formalmente che l'algoritmo garantisce un rapporto di approssimazione atteso di almeno $3/4$ [cite: 6].

Problem 4: Weighted Min-Cost Set Cover

1. Formulazione LP e Rilassamento Lineare: Introduciamo una variabile decisionale x_j per ogni insieme $S_j \in \mathcal{S}$ [cite: 6]. La formulazione intera esatta (ILP) è:

$$\min \sum_{j=1}^m c_j x_j$$

Soggetto a:

$$\sum_{j: e_i \in S_j} x_j \geq 1 \quad \forall i = 1, \dots, n$$

$$x_j \in \{0, 1\} \quad \forall j = 1, \dots, m$$

Il **rilassamento lineare** (LP) rimuove il vincolo binario permettendo valori continui [cite: 6]:

$$x_j \geq 0 \quad \forall j = 1, \dots, m$$

2. Algoritmo di Arrotondamento Deterministico: Il parametro $d = \max_i d_i$ indica il numero massimo di insiemi che contengono un singolo elemento [cite: 6]. 1. Risolviamo il problema LP rilassato per ottenere la soluzione ottima frazionaria $x^* = (x^*_1, \dots, x^*_m)$ [cite: 6]. **2. Eseguiamo il seguente arrotondamento deterministico per ogni x^*_j** [cite: 6] : * Se $x^*_j \geq \frac{1}{d}$, poniamo $\hat{x}_j = 1$ (selezioniamo l'insieme S_j). * Se $x^*_j < \frac{1}{d}$, poniamo $\hat{x}_j = 0$ (scartiamo l'insieme S_j).

3. Dimostrazione del Rapporto di Approssimazione $\leq d$: * **Ammissibilità (Copertura totale):** Consideriamo un elemento e_i . Sappiamo dal vincolo LP che $\sum_{j: e_i \in S_j} x^*_j \geq 1$. Poiché l'elemento e_i appare in al massimo d insiemi, la somma massima dei termini è $d \cdot \max_j x^*_j$. Per il principio dei cassetti, almeno uno dei termini deve soddisfare $x^*_j \geq \frac{1}{d}$. Di conseguenza, il nostro algoritmo arrotonderà a 1 la variabile per quell'insieme, assicurando che e_i sia coperto [cite: 6]. La soluzione è valida. * **Costo:** Per come abbiamo definito la regola di arrotondamento, per ogni j vale la disuguaglianza $\hat{x}_j \leq d \cdot x^*_j$. (In fatti, se $\hat{x}_j = 0$, o $\hat{x}_j = 1$ è vero; se $\hat{x}_j = 1$, sappiamo che $x^*_j \geq \frac{1}{d}$ $\implies d \cdot x^*_j \geq 1 = \hat{x}_j$). Calcoliamo il costo totale della nostra soluzione intera [cite: 6] : $\sum_{j=1}^m c_j \hat{x}_j \leq \sum_{j=1}^m c_j (d \cdot x^*_j) = d \cdot \sum_{j=1}^m c_j x^*_j = d \cdot \text{Costo}(x^*) = d \cdot \text{Costo}(OPT)$

Poiché il costo dell'ottimo frazionario $\text{Costo}(x^*)$ è un lower bound per l'ottimo intero vero ($\text{Costo}(x^*) \leq \text{Costo}(OPT)$), otteniamo [cite: 6] :

$\sum_{j=1}^m c_j \hat{x}_j \leq d \cdot \text{Costo}(OPT)$ L'algoritmo è una d -approssimazione [cite: 6].